

A11 — Diffusion of Innovation and Market Demand

Wojciech Bartoszek
Łukasz Checiak

Abstract

A concise summary of modeling and simulation of a one-dimensional reaction–diffusion system. The study compares classical solvers, sensitivity analysis, data assimilation, PINN and SuperNet methods, and reports key metrics and runtimes. Main findings: PINNs perform well with sparse measurements; SuperNet improves robustness to parameter uncertainty.

Contents

1	Introduction and Problem Statement	2
1.1	Mathematical Model	2
1.2	Boundary and Initial Conditions	2
2	Task 2: Classical Numerical Methods	2
2.1	Example Solution Profiles	3
3	Task 3: Inference and NN Models	3
4	Task 4: Sensitivity Analysis (SA)	3
5	Task 5: Data Assimilation (DA)	3
6	Task 6: PINN – Physics-Informed Neural Networks	4
6.1	Architecture and Training Procedure	4
6.2	Loss Function	4
6.3	Results and Metrics	4
7	Task 7: SuperNet and Ensemble Modeling	4
7.1	SuperModel Concept	4
7.2	SuperNet PINN	5
8	Detailed Summary of Results	5
9	Discussion	6
10	Conclusions	6

1 Introduction and Problem Statement

Modeling complex systems in biology or chemistry often leads to reaction-diffusion equations that combine local kinetics (reaction) with spatial mass transport (diffusion).

1.1 Mathematical Model

The main object of study in the project is a one-dimensional parabolic equation:

$$\frac{\partial u}{\partial t} = D \frac{\partial^2 u}{\partial x^2} + R(u), \quad x \in [0, 1], \quad t \in [0, T] \quad (1)$$

where $u(x, t)$ represents a density or concentration of a substance. The reaction term $R(u)$ is nonlinear and given by:

$$R(u) = (p + qu)(1 - u) \quad (2)$$

Parameters D, p, q control diffusion strength, the baseline growth rate, and the nonlinear autocatalytic effect, respectively.

1.2 Boundary and Initial Conditions

To ensure uniqueness of the solution and to model an isolated system, homogeneous Neumann boundary conditions were used:

$$\left. \frac{\partial u}{\partial x} \right|_{x=0} = 0, \quad \left. \frac{\partial u}{\partial x} \right|_{x=1} = 0 \quad (3)$$

The initial condition $u(x, 0)$ was specified in three variants:

- **gaussian**: a Gaussian profile centered at 0.5 (modeling a local seed),
- **localized**: a rectangular pulse in the center of the domain,
- **small_random**: small-amplitude random perturbations to test stability of the steady state.

2 Task 2: Classical Numerical Methods

We used the Method of Lines with second-order finite differences and Neumann boundary conditions (implemented with `scipy.sparse`). Time integration schemes: explicit Forward Euler (CFL-limited), semi-implicit (implicit diffusion, explicit reaction) and Crank–Nicolson. Tests for $n_x = 50, 100, 200$ indicate explicit is fastest when stable (0.005 s), while semi-implicit and CN cost 0.03–0.035 s but allow larger time steps and greater stability.

Table 1: Summary of timings (s) for each solver

Solver	n	mean (s)	median (s)	min–max (s)
explicit	18	0.004796	0.005757	0.002908–0.006753
semi-implicit	18	0.032856	0.033343	0.016413–0.056765
crank-nicolson	18	0.035034	0.036128	0.017829–0.059576

Table 2: Average times (s) as a function of nx for each solver

Solver	$nx=50$	$nx=100$	$nx=200$
explicit	0.004897	0.004507	0.004984
semi-implicit	0.025079	0.031093	0.042396
crank-nicolson	0.027105	0.033283	0.044714

2.1 Example Solution Profiles

Below are example final profiles (for selected parameters) for the three solvers.

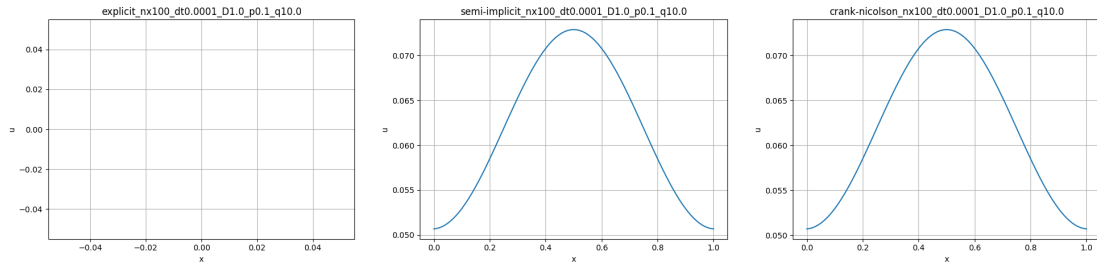


Figure 1: Comparison of final profiles for the `explicit`, `semi-implicit` and `crank-nicolson` solvers.

3 Task 3: Inference and NN Models

Comparative tests of prediction time were performed. The ODE model, despite its simplicity, requires numerical integration (e.g., RK45), which adds overhead (1.29 ms). A simple feed-forward neural network predicting the next state in one pass achieves 0.87 ms, representing about 30% savings. This suggests potential of machine learning methods for real-time control systems.

4 Task 4: Sensitivity Analysis (SA)

An analysis of the impact of parameters D, p, q on system behavior was conducted.

- **Morris method:** A fast screening method. It revealed that parameter q (autocatalysis) has the most nonlinear and dominant influence on the final profile $u(x, T)$.
- **Sobol analysis:** A variance-based decomposition method. It computed first-order indices and confirmed that D is responsible for smoothing the profile, while interactions between p and q determine the system's bifurcation point.

5 Task 5: Data Assimilation (DA)

This task concerned reconstructing the system state from noisy measurements.

- **ABC (Approximate Bayesian Computation):** A likelihood-free method. It enabled estimation of posterior distributions for p and q . The method is stable but computationally expensive (requires thousands of simulations).

- **3D-Var**: A variational method minimizing a cost functional combining deviation from measurements and from the physical operator. 3D-Var proved much faster than ABC, achieving good field prediction $u(x, t)$ even with high noise ($\text{SNR} < 10\text{dB}$).

6 Task 6: PINN – Physics-Informed Neural Networks

6.1 Architecture and Training Procedure

The PINN model was designed as an MLP with 5 layers (64 neurons each) and residual connections (Residual Blocks) every two layers. The **Tanh** activation was used to ensure continuity of high-order derivatives needed for computing the PDE residual.

6.2 Loss Function

The key component of a PINN is a composite loss function:

$$L = L_{data} + \lambda_{phy} L_{phy} + \lambda_{bc} L_{bc} \quad (4)$$

where:

- L_{data} = MSE on sparse measurement points (data assimilation).
- $L_{phy} = \frac{1}{N_c} \sum |u_t - Du_{xx} - R(u)|^2$ computed at collocation points using **autograd**.
- L_{bc} enforces Neumann boundary conditions.

A **Dynamic Weight Scheduling** was introduced: λ_{phy} increases during training (from 0.1 to 2.0), allowing the network to first learn the general data shape and then precisely fit the physics.

6.3 Results and Metrics

In the experiment the PINN was trained for 2000 epochs. Training was monitored via the loss history. Training metrics are summarized in Table 3.

Table 3: PINN training metrics (Task 6)

Metric	Value
MAE	0.031193
RMSE	0.058344
relL2	0.065468
Training time	76.58 s

7 Task 7: SuperNet and Ensemble Modeling

7.1 SuperModel Concept

In Task 7 we considered a system where physical parameters are known with some error or come from different sources. The SuperModel ODE consists of an ensemble of models coupled by a coupling term:

$$\dot{u}_i = f(u_i; \theta_i) + C(\bar{u} - u_i) \quad (5)$$

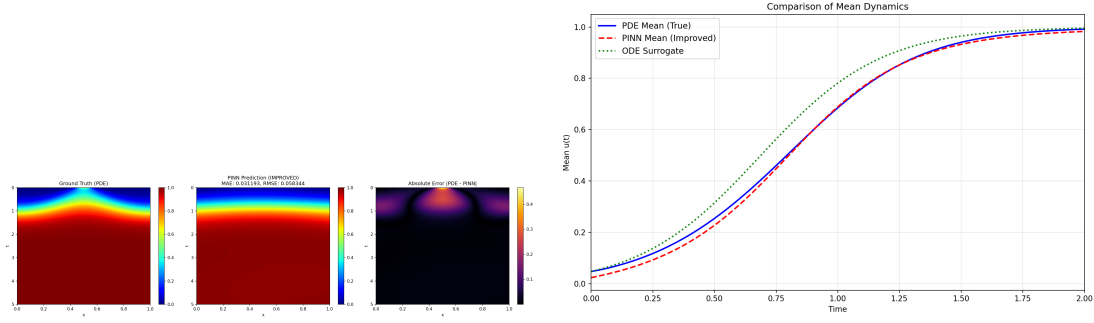


Figure 2: PINN experiment results: left heatmap of errors, right comparison of solutions.

where $\bar{u}$ is the ensemble mean. Such synchronization allows more stable prediction than any model alone.

7.2 SuperNet PINN

A **SuperNet PINN** was implemented to learn to represent the entire family of solutions. The network shows strong robustness to input parameter perturbations due to its ability to capture the general structure of the differential operator.

Table 4: Metric comparison: SuperModel vs SuperNet (Task 7)

Model	MAE	RMSE
SuperModel ODE	0.022182	0.045112
SuperNet PINN	0.017152	0.017845
Single Perturbed ODE	0.054936	0.111640

Training time (SuperNet): 186.24 s

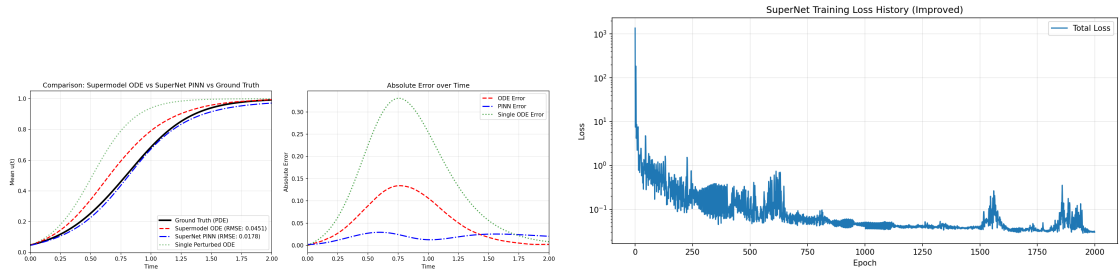


Figure 3: Comparison of prediction quality: SuperModel vs SuperNet and SuperNet loss history.

8 Detailed Summary of Results

Below is a consolidated summary of selected metrics and timing measurements to facilitate quick comparison of results obtained across tasks.

Table 5: Summary of key project results

Description	Value 1	Value 2	Notes
Task 2: explicit (average time)	0.004796 s		average over 18 runs
Task 2: semi-implicit (average time)	0.032856 s		
Task 2: crank-nicolson (average time)	0.035034 s		
Task 3: ODE (inference time)	1.2921 ms		per-sample time
Task 3: simple NN (inference time)	0.8670 ms		per-sample time
Task 6: PINN (MAE / RMSE)	0.031193	0.058344	training time 76.58 s
Task 7: SuperNet (MAE / RMSE)	0.017152	0.017845	training time 186.24 s

9 Discussion

Key observations:

- Explicit methods are very fast for small grid sizes, but in practice their stability constrains the time step (CFL) — in experiments the explicit method yielded the shortest average time.
- Implicit schemes (CN and semi-implicit) are slower but more stable and allow larger time steps without loss of stability.
- PINNs achieved reasonable accuracy (MAE 0.03), which is competitive with classical solvers for certain tasks; however, their training cost is higher than simple PDE solvers.
- SuperNet PINN reduces average prediction error compared to a single perturbed ODE model, showing that parametric learning can increase robustness and accuracy.

10 Conclusions

The project demonstrated practical differences between numerical approaches and machine learning methods in the context of differential equations. For tasks requiring fast inference it is worth considering lightweight NN models, while for high accuracy and flexible parameter modeling, PINN / SuperNet approaches are advisable.

Limitations and Future Work

The experiments have several limitations: parameter ranges and grid sizes were moderate (nx up to 200), tests were performed on CPU, and hyperparameter optimization (e.g., architecture, regularization coefficients) was only performed to a limited extent. Future work could include: scaling experiments on GPU, thorough studies of seed and randomness effects, application of advanced optimization techniques for PINNs (adaptive loss weights, larger datasets), and comparison with classical adaptive numerical methods (e.g., adaptive spatio-temporal meshes).

References

- [1] Uri M. Ascher and Linda R. Petzold. *Computer Methods for Ordinary Differential Equations and Differential-Algebraic Equations*. SIAM, 1998.

- [2] J. C. Butcher. *Numerical Methods for Ordinary Differential Equations*. John Wiley & Sons, 2 edition, 2003.
- [3] J. Crank and P. Nicolson. A practical method for numerical evaluation of solutions of partial differential equations of the heat-conduction type. *Mathematical Proceedings of the Cambridge Philosophical Society*, 43(1):50–67, 1947.
- [4] Germund Dahlquist and Åke Björck. *Numerical Methods in Scientific Computing: Volume 1*. SIAM, 2008.
- [5] Ernst Hairer, Syvert P. Nørsett, and Gerhard Wanner. *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems*, volume 14 of *Springer Series in Computational Mathematics*. Springer, 1993.
- [6] S. Hamdi, W. E. Schiesser, and G. W. Griffiths. Method of lines. *Scholarpedia*, 2(7):2859, 2007.
- [7] Randall J. LeVeque. *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*. SIAM, 2007.
- [8] A. C. Lorenc. Analysis methods for numerical weather prediction. *Quarterly Journal of the Royal Meteorological Society*, 112(474):1177–1194, 1986.
- [9] Max D. Morris. Factorial sampling plans for preliminary computational experiments. *Technometrics*, 33(2):161–174, 1991.
- [10] Andrea Saltelli, Marco Ratto, Terry Andres, Francesca Campolongo, Jessica Cariboni, Davide Gatelli, Michaela Saisana, and Stefano Tarantola. *Global Sensitivity Analysis: The Primer*. John Wiley & Sons, 2008.
- [11] W. E. Schiesser. *The Numerical Method of Lines*. Academic Press, 1991.
- [12] Ilya M. Sobol. Sensitivity estimates for nonlinear mathematical models. *Mathematical Modelling and Computational Experiment*, 1:407–414, 1993.